DETAILED PROJECT REPORT

SpendSmart

FULL-STACK PERSONAL FINANCE APPLICATION

A comprehensive guide to the project goals, user experience, application behavior, data design, API, setup, constraints, and future development.

PROJECT REPORT INCLUDES

- React 19 + Vite
- Express + Node.js
- MongoDB + Mongoose

Prepared from the SpendSmart repository | October 2026

CONTENTS / 39 PAGES

A guide to the project

This report documents the current SpendSmart implementation. Each chapter focuses on one area so the product and technical decisions can be reviewed independently.

Chapters 01-09

Executive summary; abstract; context; goals and scope; users; requirements; quality attributes; feature map; navigation and experience.

Chapters 10-18

Dashboard; transactions; categories; budgets; reports; calculations; client architecture; routing and state; shared modules.

Chapters 19-27

Server architecture; API conventions; resource endpoints; report aggregation; data models; validation; privacy and security.

Chapters 28-36

Responsive experience; installation; configuration; demo scenarios; verification plan; errors; limitations; roadmap; conclusion and glossary.

Implementation descriptions are based on the source files and README. The report distinguishes current behavior from suggested future work.

SpendSmart in one view

SpendSmart is a personal finance tracker for capturing income and expenses, organizing them into categories, planning monthly limits, and reviewing activity over time.

Core value

It gives one user a consistent workflow from recording a transaction to seeing its effect in monthly totals, category summaries, and budget progress.

Product areas

The application has five main destinations: Overview, Transactions, Budgets, Categories, and Reports.

Technology

The client uses React 19, Vite, React Router, Tailwind CSS, Recharts, Lucide React, and Axios. The server uses Node.js, Express, and Mongoose with MongoDB.

Current boundary

This is a single personal workspace. There is no login, authorization, or per-user data separation, so the current build is not intended for untrusted public use.

At a high level, the product turns records into summaries. Its accuracy depends on the user entering complete and consistent information.

Purpose and implementation summary

This project demonstrates a compact full-stack web application built around personal finance tracking. A browser interface calls a REST API; the API validates requests and stores records in MongoDB.

Problem addressed

People can have many individual purchases and income events without a clear picture of monthly cash flow or category-level spending.

Approach

SpendSmart combines transaction CRUD, custom categories, category budgets, a monthly dashboard, and server-generated reporting.

Methods

React components organize the interface into pages. Express routes expose resources. Mongoose schemas define validation and persistence.

Result

A working project structure and feature set suitable for a class demonstration, local experimentation, and further development.

This abstract describes implemented features, not an evaluated production system. No performance or user-study results are claimed.

Why organize personal finance activity

A transaction list is useful for bookkeeping, but it becomes more useful when a person can group, compare, and interpret the entries.

Scattered records

Information kept in separate notes or remembered from memory is difficult to total consistently at month end.

Limited context

A single purchase amount does not show how a category is accumulating or how it compares with an earlier month.

Need for structure

Common fields such as date, amount, type, title, and category provide a repeatable way to record financial activity.

Project response

SpendSmart stores records centrally for the local application and uses the same records to support summaries and charts.

The project focuses on personal tracking and awareness. It does not connect to banks, import statements, or provide regulated financial advice.

Define a manageable product boundary

The feature set is deliberately focused: a single user manages their own records and reviews those records by month.

Primary goals

Make entry simple, support useful organization, show progress against limits, and create clear monthly summaries.

Included scope

Transaction create/read/update/delete; search and filters; categories; monthly budgets; dashboard metrics; monthly reports; local setup.

Assumptions

The user supplies correct transaction data, uses a supported modern browser, and configures a reachable MongoDB database.

Excluded scope

Authentication, multiple profiles, bank feeds, automatic recurring entries, attachments, notifications, tax filing, and deployment operations are not implemented.

Scope boundaries help explain why the application is appropriate as a project demonstration while still needing work before broader deployment.

Designed for a personal workspace

The primary user is an individual who wants to record everyday money activity and review a monthly picture without a complex accounting workflow.

Record an expense

Enter a title, positive amount, expense type, category, date, and optional description.

Record income

Use the same transaction form with income type and an income category to include incoming funds in totals.

Plan a category

Set a budget amount for a category and month, then compare the recorded expenses with that limit.

Review a period

Choose a month and inspect net cash flow, income, expenses, category distribution, and recent expense history.

A useful class demo follows these use cases in sequence so the audience sees how each screen relies on the same underlying records.

What the application should do

The implemented requirements can be described as user-visible capabilities and corresponding system responses.

Transactions

List records; create valid entries; update an entry; delete an entry; search title, category, and description; filter by type and month.

Budgets

Create a category and month limit; list limits for a selected month; show spent, planned, remaining, and percentage used; edit and delete a limit.

Categories

Show built-in and custom category labels; filter by income or expense; create custom labels; block deletion when linked records exist.

Reports and status

Calculate monthly totals, previous-month totals, category summaries, a six-month trend, and API/database health status.

The screens provide feedback during loading and on common request failures. Data is retrieved from the configured backend API.

Qualities that shape the experience

Beyond features, the implementation includes several design choices that support clarity, maintainability, and predictable handling of common inputs.

Usability

The navigation labels are direct; page headers describe each area; empty states, progress indicators, alerts, and retry actions communicate state.

Maintainability

Pages, shared components, service helpers, finance utilities, route handlers, and data models are kept in separate modules.

Responsiveness

The application layout changes navigation and content grids across viewport breakpoints using Tailwind and custom styling.

Data correctness

Server-side validation and Mongoose validators protect key fields. Unique indexes prevent duplicate month/category budgets and duplicate typed categories.

These qualities describe implementation intent and observed source structure; they are not claims of formal usability or load testing.

How the major areas work together

The product has five main pages, shared layout components, a client service layer, four API resource groups, and three persisted record types.

Capture layer

Transactions are the primary activity records; built-in or custom categories label them as income or expense.

Planning layer

Budgets pair a category with a YYYY-MM month and positive amount. Category names connect the plan to expense records.

Insight layer

The Overview summarizes current selections; Reports aggregates a chosen month and provides a six-month expense series.

Infrastructure

Axios sends requests to Express. Express routes use Mongoose models and MongoDB; a shared error handler formats failures.

The pages are not separate databases: each summary is derived from transaction, category, and budget data maintained by the API.

A consistent shell across pages

React Router declares the application routes, while AppLayout supplies a shared navigation shell and renders the active page inside its outlet.

Primary navigation

The sidebar links to Overview, Transactions, Budgets, Categories, and Reports. On small screens, navigation becomes a compact horizontal row.

Route behavior

The root path opens the dashboard. Unknown paths redirect to the root. Page components are loaded lazily.

Page headers

PageHeader gives each view an eyebrow, title, description, and space for page-level actions or month controls.

Shared visual language

MetricCard presents a label, value, helper line, and tone-colored icon within a consistent card layout.

A shared shell lowers navigation friction and makes each feature feel like part of one product rather than a set of unrelated forms.

Read the selected month quickly

The dashboard fetches transactions and budgets, then derives its current-month totals and visual summaries in the browser.

Month selector

A native month input controls the selected month. Changing the month reloads the corresponding monthly budget list.

Headline metrics

The page shows net cash flow, income, expenses, and the combined budget limit across the selected month’s budgets.

Budget check-in

The top three budgets by percentage used are shown with progress bars, amounts, and a limit-reached message at 100 percent or more.

Visual and recent activity

Expense activity is grouped for a six-month trend and by category for the selected month; the page also surfaces recent transactions.

The dashboard derives totals from all fetched transactions on the client. It requires a working API response and a usable date on each record.

The source records behind every summary

Transactions are the central data records. The page combines summary cards, filters, an editable form, and a date-sorted history.

Fields

A transaction contains title, amount, type, category, date, and optional description. The UI initializes new entries as expenses.

Create and edit

The form checks for a non-empty title and a positive finite amount before sending. Editing pre-populates fields from the selected record.

Search and filter

Search matches title, category, and description without case sensitivity. Type and month filters can be combined.

Delete and refresh

Deletion requires a browser confirmation. After create, edit, or delete, the client reloads transactions and category options.

The server returns transaction rows sorted by descending date and identifier. The current endpoint returns the entire collection without pagination.

Use meaningful labels for grouping

The category page combines built-in labels from the frontend utility with custom category records fetched from MongoDB.

Built-in expense labels

Food, Travel, Shopping, Education, Bills, Entertainment, Health, and Other are available in the client.

Built-in income labels

Salary, Freelance, Business, Gift, and Other are available in the client.

Custom labels

A custom category stores a name, an income/expense type, and a generated normalized key. Category names are limited to 80 characters.

Safe removal

The UI lets users add and delete custom labels. The API blocks a delete or edit when transactions or budgets reference the category name.

Built-in categories are not stored as Category documents in this implementation. Custom category identity is based on type plus normalized name.

Put planned category limits beside actuals

A budget is scoped to one category and one month. Budget progress is calculated by adding matching expense transactions for that month.

Budget form

Choose a category, YYYY-MM month, and positive amount. The UI offers built-in and custom expense category choices.

Progress

For each budget, the page compares planned amount with category spending and displays a percentage and progress bar.

Summary metrics

The page sums planned amounts across budgets, sums spending for budgeted categories, and derives a combined remaining amount.

Lifecycle

Budgets can be created, edited, and deleted. A unique database index allows only one budget per category/month pair.

Budgets are a tracking and planning feature. They do not block a transaction from being recorded when spending exceeds the limit.

Server-generated comparisons and charts

Reports calls GET /api/reports/monthly with a selected YYYY-MM value and displays aggregated totals, comparisons, expense categories, and trend data.

Summary cards

Income, expenses, net cash flow, and transaction count are shown alongside comparisons with the previous month.

Expense trend

A bar chart covers the selected month and five prior months, including months with no recorded expenses.

Category share

A pie chart and list show category expense amounts for the selected month, ordered from largest amount down.

Empty and failure states

No-history messages appear when there is no data. Request errors show an explanation and a retry action.

The report endpoint uses MongoDB aggregation pipelines for totals and grouped records, rather than requiring the UI to calculate the report.

How reported amounts are derived

The project uses straightforward arithmetic to make summaries explainable. The calendar month is treated as the reporting period.

Net cash flow

For a selected month, net is income minus expenses. It may be positive, zero, or negative depending on saved records.

Category totals

Expense records are grouped by their category string; transaction count and summed amount are calculated for each group.

Budget usage

Percentage used is rounded from spent divided by limit times 100. Progress bar width is capped visually at 100 percent.

Month comparison

The report labels the percentage change relative to previous-month value. When the previous value is zero it shows new activity or no change.

Summaries inherit the accuracy and category consistency of entered records. A missing transaction will change every derived figure that depends on it.

A modular React client

The frontend is organized around route-level pages and reusable components, with API requests and finance formatting isolated from page markup.

App entry

App.jsx declares routes and uses React Suspense to show a page loader while lazy page modules are imported.

Layout

AppLayout renders the fixed/compact navigation and outlet for the selected page. PageHeader and MetricCard are shared UI elements.

Page modules

Dashboard, Transactions, Budgets, Categories, and Reports each own their data loading, local state, and user interactions.

Styling and visuals

Tailwind CSS 4 and custom CSS provide the layout; Recharts draws data visuals and Lucide React provides interface icons.

The modular structure helps keep page-specific behavior close to its page while maintaining consistency through shared components.

From route load to refreshed view

Pages use React hooks to track current data and interface state. API requests update page state and trigger React rendering.

Route entry

React Router matches a URL and renders its lazy-loaded page inside the app layout.

Loading

Pages set initial loading state, request the required data, then render a loader until the request completes.

Data state

useState holds fetched records, filters, selected month, form values, error messages, and saving flags.

Refresh after mutation

After create/update/delete operations, affected pages request current records again so displayed totals and lists stay aligned.

The client uses Promise.all for related independent reads in several views. A failed request is surfaced through page-level error feedback.

Reusable requests and finance helpers

The API service and finance utility reduce repeated plumbing and formatting logic across the route pages.

Axios client

The base URL is VITE_API_URL when configured; otherwise it uses http://localhost:5000/api. JSON is the request content type.

Resource helpers

Named functions wrap GET, POST, PUT, and DELETE calls for transactions, budgets, and categories, plus monthly report retrieval.

Currency

formatCurrency uses Intl.NumberFormat with the en-IN locale and INR currency, with a configurable maximum fraction digit count.

Date helpers

Utilities format month and date labels, derive YYYY-MM keys, and create date-input values with local timezone handling.

The helper layer gives screens a consistent request pattern while still allowing each page to decide how to present and recover from an error.

An Express API connected to MongoDB

server.js loads environment variables, configures middleware, defines health and welcome routes, mounts resource routers, and connects MongoDB before listening.

Middleware

CORS uses CLIENT_ORIGIN when set, supporting comma-separated origins; otherwise the local Vite origin is allowed. JSON request bodies are parsed.

Resource routes

Four route modules handle transactions, budgets, categories, and monthly reports under /api.

Database startup

The server requires MONGODB_URI and waits for mongoose.connect before starting the HTTP listener.

Shared errors

Unknown paths return a JSON 404. Validation, cast, and explicit 400 failures return a client error; unexpected failures return a generic 500.

The API logs failure messages on the server and avoids exposing unexpected internal error details in the response body.

Predictable JSON resource communication

The routes use JSON responses and a common success-oriented response shape for successful operations.

List response

Collection routes return success, count, and data fields. Transaction and budget lists use deterministic sorting.

Create response

A successful create returns HTTP 201 and the created record under data.

Update response

Successful updates return the updated record. A missing record returns 404; malformed object identifiers return 400.

Conflict response

Duplicate category and budget keys return 409. Linked category changes or removal also return 409 with a user-readable reason.

The endpoint list and parameters are documented in the project README. There is no API version prefix or pagination parameter in the current routes.

Validated CRUD for financial entries

The transaction router provides list, create, update, and delete operations. It validates request data before writing to MongoDB.

GET /api/transactions

Returns all transactions sorted newest first by date and then identifier; response includes the result count.

POST /api/transactions

Requires title, positive amount, valid income/expense type, and category. Optional description is limited to 500 characters and date must parse.

PUT /api/transactions/:id

Requires a valid ID and a complete valid transaction payload; applies Mongoose validators and returns the updated row.

DELETE /api/transactions/:id

Removes a matching entry and returns a success message. Missing IDs are reported as 404; malformed IDs as 400.

Amounts are converted to numbers and title/category strings are trimmed before creation or update.

Monthly limits with uniqueness protection

Budget records are filtered by month and validated around category, month format, and positive monetary value.

GET /api/budgets

Returns budgets sorted by month descending and category ascending. An optional month query narrows the list.

POST /api/budgets

Requires a category of up to 80 characters, valid YYYY-MM, and positive amount. Duplicate category/month records return 409.

PUT /api/budgets/:id

Accepts partial updates for category, month, and amount, validates changed fields, and returns the updated document.

DELETE /api/budgets/:id

Deletes a budget by valid ObjectId. A nonexistent record returns 404.

A unique compound index on category and month enforces one budget for each pair at the database layer.

Custom category management

Category documents distinguish income from expense. The server derives a normalized key from type and lowercased, trimmed name.

GET /api/categories

Lists categories by type and name; an optional valid type filter can narrow the results.

POST /api/categories

Accepts a non-empty name up to 80 characters and income/expense type. Duplicate type/name keys return 409.

PUT /api/categories/:id

Can update name or type only if there are no transactions or budgets linked by the existing category name.

DELETE /api/categories/:id

Blocks removal when records refer to the category; otherwise deletes the custom category.

Category references are stored as strings in transactions and budgets, not as MongoDB ObjectId references. The API checks those strings before edits and deletion.

Build monthly results in the database

The monthly report route validates the YYYY-MM query, creates UTC calendar boundaries, and executes aggregation queries for the requested period.

Totals pipeline

Matches transactions in the date range, groups by transaction type, and sums amounts and counts. Net is income minus expenses.

Previous month

The route calculates the preceding calendar month and runs the same totals aggregation for comparison.

Category pipeline

Matches only expenses for the selected month, groups by category, and sorts descending by summed amount.

Trend pipeline

Groups expenses by UTC YYYY-MM across six months and fills missing months with zero values in the response.

The selected month includes its start boundary and excludes the start of the following month, avoiding overlap between adjacent periods.

Represent each money event

Transaction is a Mongoose model with timestamps. A record represents one income or expense item and contains the fields used by search and summaries.

title

Required trimmed string. The route limits it to 100 characters.

amount and type

Required positive numeric amount; type is restricted to income or expense.

category and date

Required trimmed category string; date defaults to the current time if omitted.

description and timestamps

Description is optional, trimmed, and defaults to an empty string. Mongoose adds createdAt and updatedAt.

Transaction category is a name string. That keeps the model simple but means renaming a category would otherwise leave references inconsistent.

Plan limits and organize labels

Budgets and custom categories have separate Mongoose schemas that support their respective constraints and create/update timestamps.

Budget

Stores category string, YYYY-MM month string, and positive amount. The category/month pair has a unique compound index.

Category

Stores name, normalized unique key, and income/expense type. Names are trimmed and limited to 80 characters.

Derived key

A pre-validation hook builds key as type plus lowercased trimmed category name, so case variants of a same-type name conflict.

Relationship strategy

Transactions and budgets refer to category names. Category routes count linked records to protect edits and deletes.

The three collections are intentionally small and easy to explain. A future migration could use stable category identifiers if rename support is needed.

Reject invalid input close to the API

The server performs route-level checks and uses schema-level validators for important persisted data constraints.

Transaction checks

Non-empty title/category; title up to 100 characters; category up to 80; amount greater than zero; allowed type; valid optional date.

Budget checks

Valid month string, category length, and positive finite amount; duplicate pair surfaced as an explicit conflict.

Category checks

Allowed type and non-empty name up to 80 characters; duplicate normalized name/type surfaced as conflict.

Reference safety

Category edit/delete routes count transactions and budgets using the category name and reject changes while references exist.

Client-side checks improve interaction speed, but the server remains responsible for validating requests because clients can be bypassed.

Use the current build in a controlled setting

The repository documents that the current application has no user authentication or per-user data separation. All stored records belong to one shared workspace.

Access control

There are no login sessions, user identities, roles, or ownership checks in the current routes.

Network boundary

CORS limits browser origins; CORS is not authentication and does not prevent direct API requests.

Configuration

MONGODB_URI and server settings belong in local environment files. Real credentials should not be committed to source control.

Appropriate use

Use a local or test database and avoid exposing the API to untrusted users until authentication and authorization are implemented.

For a multi-user version, add identity, authorization on every data route, per-user ownership fields and indexes, secrets handling, and deployment controls.

Make core actions reachable on different screens

The interface uses responsive layouts and common semantic controls so it can adapt to narrower screens and assistive technologies.

Responsive navigation

Navigation switches between a desktop sidebar and compact horizontal links for small viewports.

Flexible content

Grid-based metric and category cards adapt to viewport width; tables and content panels use wrapping and truncation where appropriate.

Form labels

Month controls include visually hidden labels; search and filters have accessible names. Dialogs declare dialog role and modal behavior.

State feedback

Loading messages, role=alert error panels, confirmation prompts, and empty states explain results and actions.

These are implementation observations, not a formal WCAG conformance claim. A future review should include keyboard, screen reader, contrast, and zoom checks.

Prepare the local development environment

The README identifies the supported Node.js versions and a MongoDB instance as prerequisites for running the complete application.

Node.js

Use Node.js 20.19 or newer in the 20 line, or 22.12 or newer in the 22 line as specified in the project README.

MongoDB

Provide a local or hosted MongoDB database. A sample local URI is shown in server/.env.example.

Backend dependencies

From server, install the dependencies defined by package-lock.json and package.json.

Frontend dependencies

From client, install the Vite and React application dependencies using its package lock.

The frontend and backend run as separate development processes, typically in two terminals.

Connect the client to the API

Configuration is environment-based. The supplied example files provide local defaults that are useful for a development demonstration.

Server environment

Copy server/.env.example to server/.env. Set MONGODB_URI; PORT defaults to 5000 and CLIENT_ORIGIN defaults to the local Vite origin.

Start backend

Run npm run dev from server. Startup waits for a successful MongoDB connection before the server listens.

Client environment

The client defaults to http://localhost:5000/api. Set VITE_API_URL in client/.env.local to override it.

Start frontend

Run npm run dev from client and open http://localhost:5173. Check GET /api/health if the client cannot fetch data.

If the database is not reachable or MONGODB_URI is missing, the server logs a startup failure and exits rather than serving partial data.

A clear walkthrough for a class presentation

A coherent demo uses a few realistic but fictional records and demonstrates how changes propagate through the application.

Scenario A: organize

Create one custom expense category and one income category; show how the labels appear in relevant forms.

Scenario B: record

Add a small set of income and expenses with dates in the chosen month; use search, type filter, and month filter.

Scenario C: plan

Create two monthly category limits, then show the actual spent amounts and progress bars.

Scenario D: explain

Return to Overview and Reports. Connect totals and category visuals to the same records and discuss month comparisons.

Use fictional sample entries and a test database. Clean presentation data only when it is safe and appropriate for the demo environment.

How to review the project behavior

The package manifests expose development, build, and lint commands. They do not declare a dedicated automated unit or integration test framework.

Static review

Run the client lint script and inspect the server routes, models, and environment examples for consistency.

Build review

Run the Vite production build to confirm the frontend compiles into deployable static assets.

API checks

With a test database, exercise valid creates, invalid values, duplicate budget/category conflicts, missing IDs, and delete safeguards.

UI checks

Walk through each page with empty and populated data; check month filters, search, form errors, retry behavior, and narrower viewport layouts.

This report does not claim that these checks were run. They are a suggested verification plan for future project review.

Keep failures understandable and recoverable

The application provides user-facing error information for common failures and centralizes unexpected server response handling.

Connection errors

Pages explain that the API or MongoDB may be unavailable and provide a retry action where implemented.

Validation feedback

The API returns a specific message for invalid transaction, budget, category, and month inputs.

Missing records

Update and delete routes return a 404 when the requested transaction, budget, or category does not exist.

Unexpected errors

The server logs a diagnostic message and sends a generic error message for unanticipated server failures.

The current design favors clear local behavior. Operational features such as structured logs, request IDs, backups, monitoring, and rate limits are not included.

Understand what the first version does not provide

A detailed report should state the constraints alongside the features so readers can evaluate the project appropriately.

Single shared dataset

No authentication or per-user ownership means the API does not isolate one person’s records from another’s.

No bank connection

Transactions are manually entered; there is no automatic synchronization, statement import, or receipt attachment.

Unpaged lists

Transaction listing returns all records and the client filters locally. Large histories may need server-side filtering and pagination.

Category string references

Transactions and budgets refer to category names rather than stable IDs; this limits flexible rename behavior.

These are clear improvement opportunities rather than hidden capabilities. They should guide the next development stage.

A practical path for future development

The foundation supports focused improvements, provided user data protection and maintainability remain central design priorities.

First: secure access

Add authentication and authorization, persist per-user ownership, enforce it in every route, and migrate existing records safely.

Next: scale data access

Add server-side month/type/search filters and pagination; index fields used by common date and ownership queries.

Then: improve utility

Consider export, recurring entry support, and carefully designed category rename migrations.

Conclusion

SpendSmart connects simple transaction entry to category organization, budget awareness, and monthly review in a comprehensible full-stack project.

The next version should preserve clear entry and reporting while strengthening security, tests, data integrity, and operational readiness.

Terms and project files

This appendix defines a few terms used in the report and points to the main implementation locations in the repository.

CRUD and REST

CRUD means create, read, update, delete. REST is the resource-oriented HTTP API style used by the Express routes.

Aggregation

A MongoDB pipeline transforms matching records into grouped totals and trend rows without loading the full report dataset into the browser.

Main files

client/src/App.jsx routes pages; client/src/pages contains feature screens; client/src/services/api.js wraps HTTP requests.

Server files

server/server.js configures Express and MongoDB; server/routes contains API logic; server/models contains the Mongoose schemas.

Primary documentation: repository README.md and client/README.md. Project details in this report reflect the source available in the workspace at preparation time.